Declarative vs. Scripted Pipeline in Jenkins

Introduction

Jenkins Pipelines **automate CI/CD workflows** using **two pipeline syntaxes**:

1. **Declarative Pipeline** – Simplified YAML-like syntax for easy understanding.
2. **Scripted Pipeline** – More flexible but complex, written in Groovy.

- **What is the difference between Declarative and Scripted pipelines?**
- **Which one should you use for your project?**
- **How do you implement both pipelines in Jenkins?**

This guide explains **both types of Jenkins pipelines** with examples.

Section 1: Learn

What is a Declarative Pipeline?

- **Simpler structure** and **easier to read**.
- Uses **predefined blocks** (pipeline, stages, steps).
- Recommended for **beginners and standard CI/CD workflows**.

Example **Declarative Pipeline**:

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                echo 'Building...'
            }
```

```
    }
    stage('Test') {
        steps {
            echo 'Testing...'
        }
    }
    stage('Deploy') {
        steps {
            echo 'Deploying...'
        }
    }
  }
}
```

What is a Scripted Pipeline?

- **Uses full Groovy scripting** – more **flexibility and customization**.
- Can include **conditional logic, loops, and complex automation**.
- Recommended for **advanced users** with **custom CI/CD workflows**.

Example **Scripted Pipeline**:

```
node {
  stage('Build') {
    echo 'Building...'
  }
  stage('Test') {
    echo 'Testing...'
  }
```

```
    stage('Deploy') {
        echo 'Deploying...'
    }
}
```

Key Differences

Feature	Declarative Pipeline	Scripted Pipeline
Syntax	Simple and structured	Groovy-based scripting
Ease of Use	Easy for beginners	Requires coding knowledge
Flexibility	Limited customization	Highly flexible
Error Handling	Built-in error handling	Manual exception handling required
Best For	Standard CI/CD pipelines	Advanced automation scenarios

Interesting Fact

Many **large companies like Netflix and LinkedIn** use **Scripted Pipelines** for **complex automation** but **Declarative Pipelines for standard workflows**.

Section 2: Practice

1. Creating a Declarative Pipeline in Jenkins

Step 1: Open Jenkins

1. Login to **Jenkins Dashboard** → Click **New Item**.
2. Enter a **Pipeline Name** → Select **Pipeline** → Click **OK**.

Step 2: Define the Pipeline Script

1. Scroll to **Pipeline Section** → Select **Pipeline Script**.
2. Enter the following **Declarative Pipeline script**:

```
pipeline {
    agent any
    stages {
        stage('Checkout') {
            steps {
                git 'https://github.com/your-repo.git'
            }
        }
        stage('Build') {
            steps {
                sh 'mvn clean install'
            }
        }
        stage('Test') {
            steps {
                sh 'mvn test'
```

```
        }
    }
    stage('Deploy') {
        steps {
            echo 'Deploying application...'
        }
    }
  }
}
```

2. Click **Save** → **Build Now**.

Why is this useful?

- Simplifies **CI/CD workflows for easy debugging and maintenance**.

---

2. Creating a Scripted Pipeline in Jenkins

Step 1: Open Jenkins and Create a New Pipeline

Follow the same steps as above but use **Scripted Pipeline syntax**.

Step 2: Enter the Pipeline Script

```
node {
  stage('Checkout') {
      git 'https://github.com/your-repo.git'
  }
  stage('Build') {
      sh 'mvn clean install'
  }
```

```
    stage('Test') {
        sh 'mvn test'
    }
    stage('Deploy') {
        echo 'Deploying application...'
    }
}
```

Step 3: Save and Run the Pipeline

Why is this useful?

- **Allows advanced scripting features like loops and conditionals**.

---

3. Using Parallel Stages in Pipelines

- **Parallel execution speeds up build and test processes.**
- **Both declarative and scripted pipelines support parallel execution.**

Declarative Parallel Execution

```
pipeline {
    agent any
    stages {
        stage('Parallel Jobs') {
            parallel {
                stage('Job 1') {
                    steps {
                        echo 'Executing Job 1'
                    }
```

```
            }
            stage('Job 2') {
                steps {
                    echo 'Executing Job 2'
                }
            }
        }
    }
}
```

Scripted Parallel Execution

```
node {
    stage('Parallel Jobs') {
        parallel(
            Job1: {
                echo 'Executing Job 1'
            },
            Job2: {
                echo 'Executing Job 2'
            }
        )
    }
}
```

Why is this important?

- **Parallel execution reduces build times significantly**.

Section 3: Know More

Frequently Asked Questions

1. **Which pipeline should I use?**
    - **Use Declarative Pipeline** if you are new to Jenkins.
    - **Use Scripted Pipeline** if you need advanced custom logic.
2. **Can I mix Declarative and Scripted Pipelines?**
    - Yes, you can use **Scripted steps inside a Declarative Pipeline**.
3. **Where do I store pipeline scripts?**
    - Store them in a **Jenkinsfile inside a Git repository** for better version control.
4. **What happens if a pipeline fails?**
    - Check the **build logs in Jenkins** and fix the issue in your script.
5. **Can pipelines integrate with cloud services?**
    - Yes, Jenkins Pipelines can deploy applications to **AWS, Azure, and Kubernetes**.

Conclusion

Both **Declarative and Scripted Pipelines** are useful for different scenarios. Start by **writing Declarative Pipelines for simple workflows, then explore Scripted Pipelines for advanced use cases**, and soon you'll be **optimizing CI/CD automation like a pro!**